FuncMage

Domina las antiguas artes de la programación funcional

Resumen: ¡Bienvenido a FuncMage Chronicles! En el año 2142, eres un Hechicero de Funciones aprendiendo las antiguas artes de las funciones de orden superior, decoradores y hechizos lambda.

Domina estas técnicas poderosas para restaurar el equilibrio al reino digital.

Versión: 3.1

Contenido

Prólogo I **2**

II Instrucciones de IA **3**

III Introducción **5**

IV Instrucciones comunes **7**

IV.1 Requisitos de Python 7
IV.2 Importaciones autorizadas 7
IV.3 Prohibido 8
IV.4 Especificaciones de salida 8

Ejercicio 0: Santuario de Lambda **9**

VI Ejercicio 1: Reino Superior **12**

VII Ejercicio 2: Profundidades de Memoria **15**

VIII Ejercicio 3: Antigua Biblioteca **18**

IX Ejercicio 4: Torre del Maestro **21**

X Entrega y Revisión entre pares **24**

Capítulo I

Prólogo

Bienvenido, aspirante a Mago de Funciones, al año 2142!

En este futuro ciberpunk, el reino digital enfrenta el caos. Las técnicas de programación antiguas se han olvidado, y solo los Magos de Funciones poseen el conocimiento para restablecer el equilibrio. Eres uno de estos pocos escogidos, independientemente de tu origen, identidad o nivel de experiencia; cada mago aporta fortalezas únicas a nuestra diversa guild.

El Códice Lambda, un artefacto legendario que contiene los secretos de las funciones de orden superior, los decoradores y el alcance léxico, se ha esparcido por las dimensiones digitales. Como Mago de Funciones, debes dominar estas artes antiguas para reunir los fragmentos y salvar tanto el mundo virtual como el físico.

Tu viaje te llevará a través de cinco reinos místicos, cada uno enseñándote un aspecto fundamental de la programación funcional. Conocerás a otros magos de todos los ámbitos—Alex el Especialista en Lambda, Jordan el Maestro de Decoradores, Riley el Guardián del Alcance, y muchos otros que te guiarán en esta aventura.

Recuerda, joven mago: **las funciones son ciudadanos de primera clase en Python, lo que significa que** pueden ser pasadas entre sí, almacenadas en variables y transformadas igual que cualquier otro dato. Este poder, cuando se utiliza correctamente, puede crear hechizos elegantes, reutilizables y poderosos que se adaptan a cualquier situación.

Tu misión comienza ahora. ¡El reino digital espera tu contribución única al legado de los Magos de Funciones!

Capítulo II

Instrucciones de IA

Contexto

Durante tu viaje de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, acércate a ellas con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta estuvo bien formulada o de que el contenido generado es preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

Mensaje principal

- Usa IA para reducir tareas repetitivas o tediosas.
- Desarrolla habilidades de prompting (tanto de codificación como no codificación) que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar mejor y evitar riesgos comunes, sesgos y problemas éticos.
- Sigue desarrollando tanto habilidades técnicas como poderosas trabajando con tus pares.
- Utiliza solo contenido generado por IA que entiendas completamente y por el que puedas asumir la responsabilidad.

Reglas del aprendiz:

- Debes tomarte el tiempo para explorar las herramientas de IA y entender cómo funcionan, para que las uses de forma ética y reduzcas posibles sesgos.
- Debes reflexionar sobre tu problema antes de promptar; esto te ayuda a escribir prompts más claros, detallados y relevantes utilizando vocabulario preciso.
- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente cualquier cosa generada por IA.
- Siempre busca revisión de pares; no confíes únicamente en tu propia validación.

Resultados de la fase:

- Desarrolla habilidades de prompting tanto generales como específicas de dominio.
- Aumenta tu productividad con un uso efectivo de herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

Comentarios y ejemplos:

- Con frecuencia te encontrarás con situaciones exámenes, evaluaciones y más donde debes demostrar un entendimiento real. Prepárate, sigue construyendo tanto tus habilidades técnicas como interpersonales.
- Explicar tu razonamiento y debatir con tus pares a menudo revela lagunas en tu comprensión. Prioriza el aprendizaje entre pares.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a proporcionar respuestas genéricas. Tus pares, que comparten tu entorno, pueden aportar ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus pares pueden ofrecer perspectivas alternativas y matices valiosos. Confía en ellos como un punto de control de calidad.

✓ Buenas prácticas:

> Le pregunto a la IA: “¿Cómo pruebo una función de ordenamiento?” Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

✗ Malas prácticas:

> Pedir a la IA que escriba toda una función, copiarla y pegarla en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad, y fracaso en mi proyecto.

✓ Buenas prácticas:

> Uso IA para ayudar a diseñar un analizador. Luego recorro la lógica con un compañero. Enfrentamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

✗ Malas prácticas:

> Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja las tuberías. Durante la evaluación, no logro justificarlo y fracaso en mi proyecto.

Capítulo III

Introducción

Bienvenido a Crónicas FuncMage:

Has dominado los fundamentos de Python, conquistado la programación orientada a objetos y aprendido a manejar excepciones con gracia. Ahora es el momento de descubrir los paradigmas de programación más elegantes y poderosos: las técnicas de la programación funcional.

En esta aventura épica, explorarás cinco reinos místicos:

- Ejercicio 0: Sábana de Lambda - Domina las funciones anónimas y las expresiones lambda
- Ejercicio 1: Reino Superior - Descubre el poder de las funciones de orden superior
- Ejercicio 2: Profundidades de la Memoria - Comprende el alcance léxico y los cierres
- Ejercicio 3: Antigua Biblioteca - Explora los tesoros del módulo functools
- Ejercicio 4: Torre del Maestro - Crea decoradores poderosos y métodos de clase

Cada reino presenta desafíos que ampliarán tu comprensión de las capacidades funcionales de Python.

PRERREQUISITOS: Esta actividad requiere un dominio sólido de funciones de Python, clases, manejo de excepciones y estructuras de datos básicas. Debes sentirte cómodo con la sintaxis de Python y conceptos orientados a objetos antes de embarcarte en este viaje de programación funcional.

Enfoque en entender por qué los patrones de programación funcional son poderosos, no solo cómo implementarlos. Un gran Hechicero Funcional comprende la filosofía detrás de la magia.

HERRAMIENTA DE AYUDA DISPONIBLE: esta actividad incluye una herramienta auxiliar para ayudarte en tu aprendizaje. Revisa los adjuntos, extrae el archivo y busca:

- data_generator.py - te ayudará a generar datos de prueba realistas para todos los ejercicios con magos, artefactos y hechizos.

Usa python3 data_generator.py para generar datos de prueba para tus implementaciones y para probar durante la revisión.

Capítulo IV
Instrucciones comunes

IV.1 Requisitos de Python

- Su proyecto debe estar escrito en Python 3.10 o posterior.
- Su proyecto debe adherirse al estándar de codificación flake8.
- El manejo de excepciones debe proteger los flujos de datos de la corrupción.
- Use hints de tipo para todas las firmas de funciones y tipos de retorno.
- Céntrese en demostrar claramente los patrones de programación funcional.
- Cada ejercicio debe estar en su propio archivo con el nombre especificado.
- Escriba un código limpio y legible que demuestre su comprensión.

IV.2 Importaciones autorizadas

- módulo typing - para hints de tipo avanzados
- Al usar Callable como hint de tipo, usa collections.abc
- módulo itertools - para patrones de iteración avanzados
- Módulos esenciales para este proyecto **functools y operator** se especificará si permitidos en cada ejercicio.
- Todas las clases y colecciones estándar están autorizadas, junto con sus métodos (int, str, list, dict, etc.).
- Todas las funciones integradas están autorizadas, pero eval() y exec().
- Módulos de la biblioteca estándar según sea necesario para los ejercicios

IV.3 Prohibido

- Bibliotecas externas (sin instalar con pip)
- Operaciones de E/S de archivos (enfoque en procesamiento en memoria)
- Algoritmos complejos (mantener el enfoque en patrones funcionales)
- Usar eval() o exec()
- Variables globales (adopta la pureza funcional cuando sea posible)

IV.4 Especificaciones de salida

Cada ejercicio proporciona ejemplos en terminal que muestran el formato de salida sugerido. Mientras se debe mantener la estructura central, puedes personalizar los mensajes para reflejar tu comprensión de las operaciones de archivos, siempre que se conserve la información esencial.

Capítulo V

Ejercicio 0: Santuario Lambda

	Ejercicio0
ex0	
Directorio: ex0/	
Archivos para enviar: lambda_spells.py	
Autorizados: map, filter, sorted, min, max, round, sum, len	

Para este ejercicio, debes usar expresiones lambda para todas las transformaciones. No uses la palabra 'def' para crear funciones con nombre para operaciones simples, como tareas únicas o operaciones fáciles de leer. El objetivo es dominar las funciones anónimas y los patrones de programación funcional.

El Santuario Lambda: Dominando las funciones anónimas

Bienvenido al Santuario Lambda, joven mago. Aquí se enseña el antiguo arte de las funciones anónimas. Las expresiones lambda son como conjuros rápidos — hechizos cortos y poderosos que pueden transformar datos sin el ceremonial de definiciones completas de funciones.

Tu misión: Domina el arte de las expresiones lambda ayudando a a organizar artefactos mágicos usando funciones anónimas. Aprende a crear funciones al vuelo para la transformación de datos.

El desafío

Crea un archivo lambda_spells.py que contenga funciones que demuestren el dominio de las lambdas: Firmas de funciones:

```
def artifact_sorter(artifacts: list[dict]) -> list[dict] def power_filter(mages:
list[dict], min_power: int) -> list[dict] def spell_transformer(spells:
list[str]) -> list[str] def mage_stats(mages: list[dict]) -> dict
```

Requisitos de implementación

artifact_sorter(artifacts) - Ordenar artefactos mágicos:

- Usa sorted() con una lambda para ordenar por el nivel de \u0019power\u0019 (descendente)
- Cada artefacto es un dict: {\u0019name\u0019: str, \u0019power\u0019: int, \u0019type\u0019: str}
- Devolver la lista ordenada

power_filter(mages, min_power) - Filtrar magos por poder:

- Usar filter() con una lambda para encontrar magos con poder >= min_power
- Cada mago es un dict: {\u0019name\u0019: str, \u0019power\u0019: int, \u0019element\u0019: str}
- Devolver una lista de magos filtrados

spell_transformer(spells) - Transformar nombres de hechizos:

- Usar map() con una lambda para añadir el prefijo \"* \" y el sufijo \" *\"
- Entrada: lista de nombres de hechizos (cadenas)
- Devolver una lista de nombres de hechizos transformados

mage_stats(mages) - Calcular estadísticas:

- Usa lambdas con max(), min() para encontrar:
- Nivel de poder del mago más poderoso
- Nivel de poder del mago menos poderoso
- Nivel de poder promedio (redondeado a 2 decimales)
- Devolver diccionario: {\u0019max_power\u0019: int, \u0019min_power\u0019: int, \u0019avg_power\u0019: float}

Ejemplo de salida esperada:

```
$> python3 lambda_spells.py

probador de artefactos...
Bastón de Fuego (potencia 92) viene antes de Orbe de Cristal (potencia 85)

Probando transformador de hechizos...
* bola de fuego * * curar * * escudo *
```

Cómo las expresiones lambda hacen que el código sea más conciso? ¿Cuándo debería usted usar lambda frente a definiciones de funciones regulares?

Capítulo VI

Ejercicio 1: Reino Superior

	Ejercicio1
ex1	
Directorio: ex1/	
Archivos para enviar: higher_magic.py	
Autorizado: callable(), Callable	

El Reino Superior: Funciones que operan sobre funciones

Bienvenido al Reino Superior, donde las funciones se convierten en sujetos de otras funciones. Aquí aprenderás que las funciones son ciudadanos de primera clase: se pueden pasar como argumentos, devolver de otras funciones y almacenarse en estructuras de datos.

Tu misión: Ayuda al guardián del Reino, el Mago Funcional, a crear un sistema de hechizos donde las funciones pueden modificar, combinar y mejorar otras funciones. ¡Domina el arte de las funciones de orden superior!

El desafío

Crea un archivo higher_magic.py que demuestre maestría en funciones de orden superior:

Contexto

Cada hechizo en tu grimorio sigue el mismo contrato:

```
def spell(objetivo: str, poder: int) -> str
```

Un hechizo toma un objetivo y un nivel de poder y devuelve una cadena de descripción de lo ocurrido.

Ejemplos:

```
def heal(target: str, power: int) -> str: return f"Heal restores
{target} for {power} HP"
```

Si necesitas inspiración para valores de poder o objetivo, usa data_generator.py para obtener datos de muestra.

Las siguientes funciones deben ser implementadas: Firmas de funciones:

```
def spell_combiner(spell1: Callable, spell2: Callable) -> Callable def power_amplifier(base_spell:
Callable, multiplier: int) -> Callable def conditional_caster(condition: Callable, spell: Callable) ->
Callable def spell_sequence(spells: list[Callable]) -> Callable
```

Requisitos de Implementación

spell_combiner(spell1, spell2) - Combinar dos hechizos:

- Devuelve una nueva función que llame a ambos hechizos con los mismos argumentos
- El hechizo combinado debe devolver una tupla de ambos resultados
- Ejemplo: combined = spell_combiner(fireball, heal)

power_amplifier(base_spell, multiplier) - Amplificar el poder del hechizo:

- Devuelve una función con la misma firma que el hechizo original
- Devuelve un nuevo hechizo donde el poder se multiplica antes de lanzar.
- Ejemplo: mega_fireball = power_amplifier(fireball, 3)

conditional_caster(condition, spell) - Lanzar hechizo condicionalmente:

- Devuelve un nuevo hechizo que solo se lanza si la condición es verdadera.
- Si la condición falla, devuelve "Spell fizzled"
- Tanto la condición como el hechizo reciben los mismos argumentos

spell_sequence(spells) - Crear secuencia de hechizos:

- Devuelve una función que lance todos los hechizos en orden
- Cada hechizo recibe los mismos argumentos
- Devuelve una lista de todos los resultados de los hechizos

Ejemplo de salida corta:

```
$> python3 higher_magic.py

Probando combinador de hechizos...
Resultado combinado del hechizo: Bola de fuego impacta al dragón, cura al dragón

Probando amplificador de poder...
Original: 10, Amplificado: 30
```

Tu programa debe demostrar el uso y la eficiencia de todas las funciones modificadoras de tus hechizos,

¿Cómo permiten las funciones de orden superior la reutilización y la composición de código? ¿Qué hace que las funciones sean "ciudadanos de primera clase" en Python?

¿De qué paquete se recomienda usar Callable? ¿Cuál es el propósito de callable()?

Capítulo VII

Ejercicio 2: Profundidades de la Memoria

	Ejercicio2
ex2	
Directorio: ex2/	
Archivos a enviar: scope_mysteries.py	
Autorizado: nonlocal	

Las Profundidades de la Memoria: Alcance Léxico y Closures

Profundamente en las Profundidades de la Memoria, los antiguos secretos del alcance léxico están guardados. Aquí, las funciones recuerdan el entorno donde fueron creadas, capturando variables en cierres místicos que persisten más allá de su alcance original.

Tu misión: Ayuda al Guardián de la Memoria, Sabio Cierre, a entender cómo las funciones pueden "recordar" variables de su entorno de creación. Domina los cierres y el alcance léxico para crear efectos mágicos persistentes.

El Desafío

Crea un archivo scope_mysteries.py que demuestre maestría en el alcance léxico Firmas de Función:

```
def mage_counter() -> Callable def spell_accumulator(initial_power:
int) -> Callable def enchantment_factory(enchantment_type: str) ->
Callable def memory_vault() -> dict[str, Callable]
```

Requisitos de Implementación

mage_counter() - Crea un closure contador:

- Devuelve una función que cuenta cuántas veces ha sido llamada
- Cada llamada debe devolver el conteo actual (empezando desde 1)
- El contador debe persistir entre llamadas
- Crear dos contadores separados debe producir estados independientes.
- Usa closure para mantener el estado sin variables globales

spell_accumulator(initial_power) - Crea un acumulador de poder:

- Devuelve una función que acumula poder con el tiempo
- Cada llamada suma la cantidad dada al poder total
- Devuelve el nuevo poder total después de cada suma
- Comienza con initial_power como base

enchantment_factory(enchantment_type) - Crea funciones de encantamiento:

- Devuelve una función que aplica el encantamiento especificado
- La función devuelta toma el nombre de un objeto y devuelve la descripción encantada
- Formato: "enchantment_type item_name" (ej., "Espada Llameante")
- Cada fábrica crea funciones con diferentes tipos de encantamiento

memory_vault() - Crea un sistema de gestión de memoria:

- Devuelve un diccionario con funciones "store" y "recall"
- Función "store": toma (clave, valor) y almacena la memoria
- Función "recall": toma (clave) y devuelve el valor almacenado o "Memoria no encontrada"
- Usa closure para mantener el almacenamiento de memoria privado

Ejemplo de salida esperada:

```
$> python3 scope_mysteries.py
Probando contador de magos...
counter_a llamada 1: 1
counter_a llamada 2: 2
counter_b llamada 1: 1

Probando acumulador de hechizos...
Base 100, añadir 20: 120
Base 100, añadir 30: 150

Probando fábrica de encantamientos...
Espada llameante
Escudo congelado

Probando bóveda de memoria...
Almacenar 'secret' = 42
Recordar 'secret': 42
Recordar 'unknown': Memoria no encontrada
```

¿Cómo permiten las clausuras que las funciones "recuerden" su entorno de creación? ¿Cuáles son los beneficios del alcance léxico en la programación funcional?

¿Por qué existen diferencias globales? prohibido, pero ¿no local permitido? ¿Cuáles son las claves?

Capítulo VIII

Ejercicio 3: Biblioteca Antigua

	Ejercicio3
ex3	
Directorio: ex3/	
Archivos a entregar: functools_artifacts.py	
Autorizado: functools, operator	

La Biblioteca Antigua: Tesoros de Functools

En la Biblioteca Antigua, se guardan los artefactos de programación funcional más potentes. El módulo functools contiene herramientas legendarias: reduce, partial, wraps y más.
Estos artefactos pueden transformar la forma en que abordas problemas complejos.

Tu misión: Ayuda al curador de la Biblioteca, Archivista Functools, catalogar y demostrar el poder de estos artefactos antiguos. ¡Aprende a manejar reduce, partial y otros tesoros de functools!

El desafío

Crea un archivo functools_artifacts.py que demuestre la maestría de functools Firmas de Función:

```
def spell_reducer(spells: list[int], operation: str) -> int def
partial_enchanter(base_enchantment: Callable) -> dict[str, Callable] def
memoized_fibonacci(n: int) -> int def spell_dispatcher() -> Callable[[Any], str]
```

Requisitos de Implementación

spell_reducer(spells, operation) - Reduce poderes de hechizo:

- Usa functools.reduce para combinar todos los poderes de hechizo
- Soporta operaciones: "add", "multiply", "max", "min"
- Usa las funciones del módulo operator (add, mul, etc.)
- Devuelve el valor final reducido
- Si los hechizos están vacíos, devuelve 0
- Si la operación es desconocida, maneja el error adecuadamente

partial_enchanter(base_enchantment) - Crear aplicaciones parciales:

- Toma una función de encantamiento base con la firma (power: int, element: str, target: str) -> str
- Usa functools.partial para crear 3 versiones especializadas
- Cada versión prellena power=50 y el elemento

memoized_fibonacci(n) - Fibonacci cacheada:

- Usa el decorador functools.lru_cache para memoización
- Implementa el cálculo de la secuencia de Fibonacci
- La función debe devolver el n-ésimo número de Fibonacci
- La memoria caché debe mejorar el rendimiento para llamadas repetidas
- Devuelve el n-ésimo número de Fibonacci

Puedes verificar que la caché funciona mediante memoized_fibonacci.cache_info().

spell_dispatcher() - Crear un sistema de despacho único:

- Usa el decorador functools.singledispatch para crear un sistema de hechizos
- La función base recibe Any y maneja tipos de hechizo desconocidos
- Maneja diferentes tipos: int (hechizo de daño), str (encantamiento), list (multi-lanzamiento)
- Devuelve la función de despacho
- Cada tipo debe tener un comportamiento de hechizo apropiado

Ejemplo de salida esperada:

```
$> python3 functools_artifacts.py

Probando reductor de hechizos...
Suma: 100
Producto: 240000
Máximo: 40

Probando fibonacci memoizado...
Fib(0): 0
Fib(1): 1
Fib(10): 55
Fib(15): 610

Probando el despachador de
hechizos... Hechizo de daño: 42 daño
Encantamiento: bola de fuego
Multiprecast: 3 hechizos Tipo de
hechizo desconocido
```

Cómo ¿cómo reduce functools.reduce habilita una poderosa agregación de datos? ¿Qué cuáles son los beneficios de rendimiento de la memoización con lru_cache?

Capítulo IX

Ejercicio 4: Torre del Maestro

	Ejercicio4
ex4	
Directorio: ex4/	
Archivos a entregar: decorator_mastery.py	
Autorizado: functools.wraps, staticmethod	

La Torre del Maestro: Dominio de Decoradores y Métodos de Clase

En la cúspide de tu viaje se encuentra la Torre del Maestro, donde los magos de Función más avanzados aprenden a crear decoradores—envoltorios mágicos que pueden transformar el comportamiento de cualquier función. Aquí, también dominarás @staticmethod y entenderás cómo funcionan los decoradores con las clases.

Tu misión: Demuestra tu dominio ante el guardián de la Torre, Gran Maestro Decorador, creando decoradores potentes que pueden mejorar cualquier conjuro o método. ¡Este es tu examen final como Mago de Funciones!

El desafío

Crea un archivo decorator_mastery.py que demuestre dominio de decoradores:
Firmas de funciones:

```
def spell_timer(func: Callable) -> Callable def
power_validator(min_power: int) -> Callable def
retry_spell(max_attempts: int) -> Callable

Gremio de Magos:
   @staticmethod
   def validate_mage_name(name: str) -> bool
   def cast_spell(self, spell_name: str, power: int) -> str
```

Requisitos de implementación

spell_timer(func) - Decorador de tiempo de ejecución:

- Crea un decorador que mida el tiempo de ejecución de la función
- Imprimir "Casting function_name..." antes de la ejecución
- Imprimir "Spell completed in X.XXX seconds" después de la ejecución (3 decimales)
- Usar functools.wraps para conservar los metadatos de la función original
- Devuelve el resultado de la función original

power_validator(min_power) - Decorador de validación parametrizado:

- Crea una fábrica de decoradores que valide los niveles de potencia
- Aplicado a una función independiente cuyo primer argumento es la potencia.
- Si la potencia es válida (>= min_power), ejecuta la función normalmente
- Si es inválida, devuelve "Insufficient power for this spell"
- Usa functools.wraps correctamente

retry_spell(max_attempts) - Decorador de reintentos:

- Crea un decorador que reintente hechizos que fallen
- Si la función lanza una excepción, reintentar hasta max_attempts veces
- Imprimir "Spell failed, retrying... (attempt n/max_attempts)"
- Si todos los intentos fallan, devuelve "Spell casting failed after max_attempts attempts"
- Si un intento tiene éxito, devuelve su resultado normalmente

Clase MageGuild - Demuestra staticmethod:

- validate_mage_name(name) - Método estático que verifica si el nombre es válido
- El nombre es válido si tiene al menos 3 caracteres y contiene solo letras y espacios
- cast_spell(self, spell_name, power) - Método de instancia
- Debe usar el decorador power_validator con min_power=10
- Cuando la potencia es válida, devuelve "Successfully cast spell_name with <power> power"
- De lo contrario devuelve "Insufficient power for this spell"

Puedes usar time.sleep para simular el tiempo de espera entre hechizos.

Ejemplo de salida esperada:

```
$> python3 decorator_mastery.py
Probando temporizador de hechizos...
Lanzando bola de fuego...
Hechizo completado en 0,101 segundos
Resultado: ¡Bola de fuego lanzada!

Probando hechizo de nuevo...
El hechizo falló, intentándolo de nuevo... (intento 1/3)
El hechizo falló, intentándolo de nuevo... (intento 2/3)
La emisión de hechizo falló tras 3 intentos
¡GagáWAaaagh escrito!

Probando MageGuild...
Verdadero
Falso
Encantamiento exitoso de rayo con 15 poder
Potencia insuficiente para este hechizo
```

¿Cómo permiten los decoradores la separación de responsabilidades? ¿Cuál es la diferencia entre @staticmethod y métodos de instancia regulares?

Capítulo X

Envío y revisión por pares

Envía tu tarea en tu repositorio Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la revisión por pares. No dudes en verificar dos veces los nombres de tus archivos para asegurar que sean correctos.

Durante la revisión por pares, podría pedirle que explique conceptos de programación funcional, demuestre cómo funcionan los cierres o muestre cómo los decoradores transforman las funciones. Concéntrese en entender los conceptos, no solo la implementación.

Mantenga sus implementaciones centradas en demostrar claramente los conceptos de programación funcional. Evite sobreingenierizar: el objetivo es demostrar maestría de funciones de orden superior, cierres y decoradores.

¡Felicidades, Mago de Funciones! Has dominado las antiguas artes de la programación funcional. Estas técnicas harán que tu código sea más elegante, reutilizable y poderoso. ¡Úsalas sabiamente en tus futuras aventuras!